Spring Boot

1、JsonResult工具类

2、定时任务

3、异步任务

4、使用 Spring Boot 默认的数据源：HicarCP

5、整合 MyBatis 的配置

6、SpringBoot 配置文件

7、@RequestBody

8、@Transactional注解的可用参数

- readOnly
- rollbackFor
- rollbackForClassName
- noRollbackFor
- noRollbackForClassName
- timeout
- propagation
- isolation
- 什么是脏读、幻读、不可重复读?

9、后端允许跨域

10、

11、

12、

13、

14、

15、

16、

17、

18、

19、

20、

1、JsonResult工具类

JsonResult工具类 Java 复制代码

```java
/*
    使用@RestController或@ResponseBody时，可直接返回该对象，
    Spring Boot默认使用Jackson会自动将该对象转换为json字符串
    如{"code": 0,"msg": "","data": [{}, {}]}
 */
public class JsonResult extends HashMap<String, Object> implements Serializable {
    public static final long serialVersionUID = 1L;
    // 成功
    public static final int SUCCESS = 0;
    // 失败
    public static final int FAIL = 1;
    // 错误
    public static final int ERROR = 2;
    // 未登录
    public static final int LOGOUT = 1001;

    public JsonResult(int code, String msg, Object data) {
        super(3);  //继承自Map，设置初始容量
        this.put("code", code); //状态码，layui中code=0表示成功
        this.put("msg", msg);   //提示消息
        this.put("data", data); //数据体
    }

    //一般返回code、msg和data这三个即可，但layui加载table时还要求count值
    //添加额外的返回值
    public JsonResult put(String key, Object value) {
        super.put(key, value);
        return this;
    }

    //快速返回请求成功
    public static JsonResult success(Object data) {
        return new JsonResult(SUCCESS, "ok", data);
    }

    //快速返回请求失败
    public static JsonResult fail(String msg) {
        return new JsonResult(FAIL, msg, null);
    }

    public static JsonResult error(String msg, Object data) {
        return new JsonResult(ERROR, msg, data);
    }

```

```java
    public static JsonResult logout() {
        return new JsonResult(LOGOUT, "未登录", null);
    }

    //快速生成一个Map键值对
    public static Map<String, Object> fastMap(String key, Object value) {
        Map<String, Object> data = new HashMap<>(1);
        data.put(key, value);
        return data;
    }
}
```

- JsonResult工具类测试

JsonResult工具类测试 Java 复制代码

```java
// 将方法的返回值直接作为响应报文的响应体响应到浏览器；表明是 Controller 层
@RestController
// 注入日志类，以后不用在控制台 打印
@Slf4j
public class JsonResultController {

    // JsonResult 测试
    @PostMapping("/getJsonResult")
    public JsonResult getJsonResult(){

        Map<Integer,String> count = new HashMap<>();
        count.put(1,"1");
        count.put(2,"2");

        List<String> list = new ArrayList<>();

        list.add("1111");
        list.add("2222");

        for (String s : list) {
            log.info(s);
        }

        return JsonResult.success(list).put("count", count);
    }

}
```

2、定时任务

> 主类上使用 `@EnableScheduling` 注解开启对定时任务的支持

定时任务 Java 复制代码

```java
// 注入日志类，以后不用在控制台 打印
@Slf4j
// 1、标记配置类，使得 Spring Boot 容器可以扫描到
@Configuration
// 2、开启定时任务
@EnableScheduling
public class MyTask {

    // 3、添加一个任务，并且注明任务的运行表达式
    @Scheduled(cron = "*/5 * * * * ?")
    public void publishMsg(){
        log.warn("开始执行任务: " + LocalDateTime.now());
    }

}
```

```
//      常用的定时任务表达式:
//      每隔5秒执行一次: */5 * * * * ?
//      每隔1分钟执行一次: 0 */1 * * * ?
//      每天23点执行一次: 0 0 23 * * ?
//      每天凌晨1点执行一次: 0 0 1 * * ?
//      每月1号凌晨1点执行一次: 0 0 1 1 * ?
//      每月最后一天23点执行一次: 0 0 23 L * ?
//      每周星期天凌晨1点实行一次: 0 0 1 ? * L
//      在26分、29分、33分执行一次: 0 26,29,33 * * * ?
//      每天的0点、13点、18点、21点都执行一次: 0 0 0,13,18,21 * * ?
```

3、异步任务

异步任务 Java 复制代码

```java
// 注入日志类，以后不用在控制台 打印
@Slf4j
// 1、标记配置类，使得 Spring Boot 容器可以扫描到
@Configuration
// 2、开启定时任务
@EnableAsync
public class AsyncTask {

    // 3、添加一个异步任务
    @Async
    public void publishMsg(){
        try {
            // 休眠 5 秒钟
            Thread.sleep(5000);
            log.warn("开始执行异步：" + LocalDateTime.now());
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

}
```

调用异步任务 Java 复制代码

```java
// 将方法的返回值直接作为响应报文的响应体响应到浏览器；表明是 Controller 层
@RestController
// 注入日志类，以后不用在控制台 打印
@Slf4j
public class HelloController {

    // 自动注入
    @Autowired
    private AsyncTask asyncTask;

    // 请求地址，处理用户的 /hello 请求
    @RequestMapping("/hello")
    public String hello(){
        // 运行异步任务
        asyncTask.publishMsg();

        // 测试是否会跳过异步任务来执行其后面的语句
        log.info("测试是否会跳过异步任务来执行其后面的语句");

        return "Hello Spring Boot 2";
    }
}
```

4、使用 Spring Boot 默认的数据源：HicarCP

yml配置文件 YAML 复制代码

```yaml
# yaml 配置文件
spring:
  datasource:
    # 指定数据源的类型：HikariDataSource
    type: com.zaxxer.hikari.HikariDataSource
    # 连接信息一定要用双引号引起来
    # characterEncoding=UTF-8：指定所处理字符的解码和编码的格式
    # useSSL=false：不进行 SSL 连接
    # allowPublicKeyRetrieval=true：允许恶意代理执行MITM攻击以获取明文密码，因此它在默认情况下为False，必须显式启用
    # serverTimezone=GMT：格林尼治标准时间
    url: "jdbc:mysql://127.0.0.1:3306/sqlTest?characterEncoding=UTF-8&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=GMT"
    username: "root"
    password: "000123"
    # 数据库驱动名
    driver-class-name: com.mysql.cj.jdbc.Driver
    # 数据源配置
    hikari:
      # 等待连接池分配连接的最大时长（毫秒），超过这个时长还没可用的连接则发生SQLException，默认 30 秒
      connection-timeout: 30000
      # 最小连接数
      minimum-idle: 5
      # 最大连接数
      maximum-pool-size: 20
      # 自动提交
      auto-commit: true
      # 连接超时的最大时长（毫秒），超时则被释放（retired），默认 10 分钟
      idle-timeout: 600000
      # 连接池名字
      pool-name: DateSourceHikariCP
      # 连接的生命时长（毫秒），超时而且没被使用则被释放（retired），默认 30 分钟，1800000ms
      max-lifetime: 1800000
      # 连接时发起的测试 sql 脚本
      connection-test-query: SELECT 1
```

5、整合 MyBatis 的配置

整合 MyBatis 的配置 YAML 复制代码

```yaml
# 配置 MyBatis 规则
mybatis:
  # 所有数据表逆向后所一一映射的实体类
  type-aliases-package: com.yuehai.bean
  # 全局配置文件位置
  # config-location: "classpath:mybatis/mybatis-config.xml"
  # sql 映射文件位置
  mapper-locations: "classpath:mybatis/mapper/*.xml"
  # 配置 MyBatis 的配置项，一旦使用这个方式，就不能再配置上面的：config-location
  # 所以也可以不写（创建）全局配置文件，所有全局配置文件的配置都放在configuration配置项中即可
  configuration:
    # 是否开启自动驼峰命名规则映射
    map-underscore-to-camel-case: true
    # 开启 mybatis 的 sql 执行日志打印，可以在控制台打印输出 sql 语句
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl

# 通用 mapper 配置
mapper:
  # 所有 mapper 都需要实现的接口
  mappers: com.yuehai.mapper.MyMapper
  # 在进行数据库操作的时候，判断一个属性是否为空的时候，是否需要自动追加不为空字符串的判断；如：username != null 不需要 username != ''
  not-empty: false
  # 身份标识
  i-d-e-n-t-i-t-y: MYSQL

# 分页插件助手的配置
pagehelper:
  # 身份标识
  helper-dialect: mysql
  # 支持方法中的一些参数
  support-methods-arguments: true
```

6、SpringBoot 配置文件

SpringBoot 配置文件 YAML 复制代码

```yaml
# yaml 配置文件
spring:
  datasource:
    # 指定数据源的类型：HikariDataSource
    type: com.zaxxer.hikari.HikariDataSource
    # 连接信息一定要用双引号引起来
    # characterEncoding=UTF-8：指定所处理字符的解码和编码的格式
    # useSSL=false：不进行 SSL 连接
    # allowPublicKeyRetrieval=true：允许恶意代理执行MITM攻击以获取明文密码，因此它在默认情况下为False，必须显式启用
    # serverTimezone=GMT：格林尼治标准时间
    url: "jdbc:mysql://127.0.0.1:3306/sqlTest?characterEncoding=UTF-8&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=GMT"
    username: "root"
    password: "000123"
    # 数据库驱动名
    driver-class-name: com.mysql.cj.jdbc.Driver
    # 数据源配置
    hikari:
      # 等待连接池分配连接的最大时长（毫秒），超过这个时长还没可用的连接则发生SQLException，默认 30 秒
      connection-timeout: 30000
      # 最小连接数
      minimum-idle: 5
      # 最大连接数
      maximum-pool-size: 20
      # 自动提交
      auto-commit: true
      # 连接超时的最大时长（毫秒），超时则被释放（retired），默认 10 分钟
      idle-timeout: 600000
      # 连接池名字
      pool-name: DateSourceHikariCP
      # 连接的生命时长（毫秒），超时而且没被使用则被释放（retired），默认 30 分钟，1800000ms
      max-lifetime: 1800000
      # 连接时发起的测试 sql 脚本
      connection-test-query: SELECT 1
  mvc:
    # 开启页面表单的 Rest 功能
    hiddenmethod:
      filter:
        enabled: true

# 配置 MyBatis 规则
mybatis:
```

```yaml
  # 全局配置文件位置
  # config-location: "classpath:mybatis/mybatis-config.xml"
  # sql 映射文件位置
  mapper-locations: "classpath:mapper/*.xml"
  # 配置 MyBatis 的配置项，一旦使用这个方式，就不能再配置上面的：config-location
  # 所以也可以不写（创建）全局配置文件，所有全局配置文件的配置都放在configuration配置项中即可
  configuration:
    # 是否开启自动驼峰命名规则映射
    map-underscore-to-camel-case: true

# 通用 mapper 配置
mapper:
  # 所有 mapper 都需要实现的接口
  # mappers: com.yuehai.mapper.MyMapper
  # 在进行数据库操作的时候，判断一个属性是否为空的时候，是否需要自动追加不为空字符串的判断；如：username != null 不需要 username != ''
  not-empty: false
  # 身份标识
  i-d-e-n-t-i-t-y: MYSQL

# 分页插件助手的配置
pagehelper:
  # 身份标识
  helper-dialect: mysql
  # 支持方法中的一些参数
  support-methods-arguments: true
```

7、@RequestBody

1. 接收请求体（JSON字符串），映射到对象上，不能是单个的基本数据类型（与其包装类）
2. 若想获取单个的基本数据类型，应使用：
    a. @RequestParam：获取请求参数

html 页面的超链接 HTML 复制代码

```html
<a href="/getTest3?id=1&name=yuehai">car</a>
```

controller层代码 Java 复制代码

```java
// RESTful 风格的 URI，请求方式为：GET，查询
@GetMapping("/getTest3")
// @RequestParam("id")：获取名为 id 的请求参数，并赋值给后面定义的参数
// @RequestParam：获取所有请求参数，以键值对的方式方式赋值给集合
public Map<String,Object> getTest3(@RequestParam("id") Integer id,
                                   @RequestParam("name") String name,
                                   @RequestParam Map<String,String> param
){
    Map<String,Object> map = new HashMap<>();
    map.put("id",id);
    map.put("name",name);
    map.put("param",param);

    // 结果：{"param":{"id":"1","name":"yuehai"},"name":"yuehai","id":1}
    return map;
}
```

b. @PathVariable：路径变量

html 页面的超链接 HTML 复制代码

```html
<a href="/getTest/1/yuehai">car/{id}/{username}</a>
```

controller层代码 Java 复制代码

```java
// RESTful 风格的 URI，请求方式为：GET，查询
@GetMapping("/getTest/{id}/{name}")
// @PathVariable：路径变量：将请求参数中 id 的值赋值给定义的参数 id
public Map<String,Object> getTest(@PathVariable("id") Integer id,
        @PathVariable("name") String name,
        // 若是 map 集合，则会将所有数据封装到键值对中
        @PathVariable Map<String,String> pv){
    Map<String,Object> map = new HashMap<>();
    map.put("id",id);
    map.put("name",name);
    map.put("pv",pv);

    // 结果：{"pv":{"name":"yuehai","id":"1"},"name":"yuehai","id":1}
    return map;
}
```

c. @RequestBody：获取请求体（需请求方式为POST），使用对象接收，只接收对应的一个属性

html 页面的表单（或JSON字符串） HTML 复制代码

```html
<form action="/getTest5" method="post">
    测试@RequestBody获取数据 <br/>
    用户名：<input name="userName"/> <br>
    邮箱：<input name="email"/>
    <input type="submit" value="提交"/>
</form>
```

controller层代码 Java 复制代码

```java
 // RESTful 风格的 URI，请求方式为：Post，添加
 @PostMapping("/getTest5")
 // @RequestBody：获取请求体（需请求方式为POST），即表单内容
 public Map<String,Object> getTest5(@RequestBody String content){
     Map<String,Object> map = new HashMap<>();
     map.put("content",content);

     // 结果：{"content":"userName=yuehai&email=123"}
     return map;
 }
```

3. 若想获取多个参数（JSON），应使用：@RequestBody
   a. 若是参数可以与一个对象一一对应，或者参数少于对象属性（但是必须可以对应），可以使用对象接收参数

使用对象接收参数 Java 复制代码

```java
// 修改，根据 id 修改
@PutMapping("/user")
public JsonResult updateUser(@RequestBody User user){
    // 修改，根据 id 修改
    Boolean success = userService.updateUserById(user);

    return JsonResult.success(user);
}
```

   b. 若是参数太多不能与对象一一对应，可以使用 Map 集合接收参数

使用 Map 集合接收参数 Java 复制代码

```java
// 模拟转账；根据 id 修改
@PutMapping("/user/transferAccounts")
public JsonResult transferAccounts(@RequestBody Map<String,Integer> map){
    // 调用转账 service
    Boolean success = userService.transferAccounts(map.get("outId"), map.get("collectId"), map.get("money"));

    return JsonResult.success(map.get("money"));
}
```

8、@Transactional注解的可用参数

readOnly

该属性用于设置当前事务是否为只读事务，设置为true表示只读，false则表示可读写，默认值为false

rollbackFor

该属性用于设置需要进行回滚的异常类数组，当方法中抛出指定异常数组中的异常时，则进行事务回滚。例如： 1. 指定单一异常类：@Transactional(rollbackFor=RuntimeException.class) 2. 指定多个异常类：@Transactional(rollbackFor={RuntimeException.class, BusnessException.class})

rollbackForClassName

该属性用于设置需要进行回滚的异常类名称数组，当方法中抛出指定异常名称数组中的异常时，则进行事务回滚。例如： 1. 指定单一异常类名称：@Transactional(rollbackForClassName="RuntimeException") 2. 指定多个异常类名称：@Transactional(rollbackForClassName={"RuntimeException","BusnessException"})

noRollbackFor

该属性用于设置不需要进行回滚的异常类数组，当方法中抛出指定异常数组中的异常时，不进行事务回滚

noRollbackForClassName

参照上方的例子

timeout

该属性用于设置事务的超时秒数，默认值为-1表示永不超时

propagation

该属性用于设置事务的传播行为 例如：@Transactional(propagation=Propagation.NOT_SUPPORTED)

事物传播行为介绍:

1. @Transactional(propagation=Propagation.REQUIRED) 如果有事务, 那么加入事务, 没有的话新建一个(默认)
2. @Transactional(propagation=Propagation.NOT_SUPPORTED) 容器不为这个方法开启事务
3. @Transactional(propagation=Propagation.REQUIRES_NEW) 不管是否存在事务,都创建一个新的事务,原来的挂起,新的执行完毕,继续执行老的事务
4. @Transactional(propagation=Propagation.MANDATORY) 必须在一个已有的事务中执行,否则抛出异常
5. @Transactional(propagation=Propagation.NEVER) 必须在一个没有的事务中执行,否则抛出异常(与Propagation.MANDATORY相反)
6. @Transactional(propagation=Propagation.SUPPORTS) 如果其他bean调用这个方法,在其他bean中声明事务,那就用事务.如果其他bean没有声明事务,那就不用事务

isolation

该属性用于设置底层数据库的事务隔离级别

事务隔离级别介绍:

1. @Transactional(isolation = Isolation.READ_UNCOMMITTED)读取未提交数据(会出现脏读, 不可重复读)基本不使用
2. @Transactional(isolation = Isolation.READ_COMMITTED)读取已提交数据(会出现不可重复读和幻读)
3. @Transactional(isolation = Isolation.REPEATABLE_READ)可重复读(会出现幻读)
4. @Transactional(isolation = Isolation.SERIALIZABLE)串行化

什么是脏读、幻读、不可重复读?

1. 脏读：一个事务读取到另一事务未提交的更新数据
2. 不可重复读：在同一事务中, 多次读取同一数据返回的结果有所不同, 换句话说, 后续读取可以读到另一事务已提交的更新数据. 相反, "可重复读"在同一事务中多次读取数据时, 能够保证所读数据一样, 也就是后续读取不能读到另一事务已提交的更新数据
3. 幻读：一个事务读到另一个事务已提交的insert数据

其中MySQL默认使用的隔离级别为REPEATABLE_READ、Oracle的为READ_COMMITTED

9、后端允许跨域

- 在 Controller 类或者方法上添加注解： `@CrossOrigin`

Java | 复制代码

```java
package com.yuehai.controller;

import org.springframework.web.bind.annotation.*;

/**
 * @author 10222148
 * @create 2022/11/23 13:10
 */

@CrossOrigin
@RestController
public class AxiosController {

    @GetMapping("/hello")
    public String getHello() {
        return "Hello Spring Boot 2 GET";
    }

    @PostMapping("/hello")
    public String postHello() {
        return "Hello Spring Boot 2 POST";
    }

    @DeleteMapping("/hello")
    public String deleteHello() {
        return "Hello Spring Boot 2 DELETE";
    }

    @PutMapping("/hello")
    public String putHello() {
        return "Hello Spring Boot 2 PUT";
    }
}

```

10、

11、

12、

13、

14、

15、

16、

17、

18、

19、

20、